

ENUNCIADO TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Fecha límite de entrega: 8 de Septiembre de 2026

Equipo ALGORITMOS Y COMPLEJIDAD 2026-2

30 de agosto de 2026

18:47

Versión 0.0

Índice

1. Objetivos	3
2. Especificaciones	3
2.1. Implementaciones	3
2.2. Mini-informe	5
3. Condiciones de entrega	5
A. Casos de prueba	6
A.1. Ordenamiento de un arreglo unidimensional de números enteros	6
A.2. Multiplicación de matrices cuadradas de números eneteros	6

1. Objetivos

El objetivo de esta tarea es introducirnos en el Análisis experimental de Algoritmos. Para ello, se propone realizar un estudio experimental de 4 algoritmos de ordenamiento y 2 de multiplicación de matrices.

Para cada uno de los dos problemas, el de ordenar un arreglo unidimensional de números enteros, y el de multiplicar dos matrices cuadradas, existen múltiples algoritmos que los resuelven, y la mayoría, si no todos, tienen implementaciones de fácil acceso (ya se encuentran en bibliotecas de lenguajes de programación, o son fáciles de encontrar en Internet). Sus complejidades teóricas son conocidas, ya que éstas son una característica fundamental a la hora de diseñar algoritmos y, en la mayoría de los casos, la motivación para diseñarlos.

El desafío en esta tarea no está en diseñar los algoritmos (los encuentra fácilmente), ni tampoco en calcular o demostrar su complejidad teórica: el desafío está en **estudiar su comportamiento en la práctica** y relacionar este comportamiento con la complejidad teórica.

2. Especificaciones

En esta sección se describen los pasos a seguir para completar la tarea, la cual consiste en desarrollar código en C++ «[Implementaciones](#)» y en realizar un mini-informe «[Mini-informe](#)». Se espera que cada uno de los pasos se realice de manera ordenada y siguiendo las instrucciones dadas.

2.1. Implementaciones

Nota: Abra este documento en algún lector de pdf que permita hipervínculos, ya que en este documento el texto en color azul suele indicar un hipervínculo.

- (1) En caso de dudas, enviar preguntas **al foro de la Tarea 1**. En caso de cualquier de modificaciones, todas se informarán por aula.
- (2) Todo lo necesario para realizar la tarea se encuentra en el archivo .zip proporcionado a través de aula.usm.cl.

Nota: La estructura de archivos y el template del proyecto se encuentran disponibles en el material entregado.

(1) **Ordenamiento y Multiplicación.**

Ordenamiento. Implementar en C++ los algoritmos de ordenamiento: MERGE SORT, QUICK SORT, PATIENCE SORT y el algoritmo SORT de la librería estándar de C++. Use la siguiente estructura de archivos:

- code/sorting/algorithms/mergesort.cpp
- code/sorting/algorithms/quicksort.cpp
- code/sorting/algorithms/patiencesort.cpp
- code/sorting/algorithms/sort.cpp

El algoritmo SORT ya se encuentra implementado en el material entregado con el fin de que sea incluido en sus mediciones del punto (2) de la subsección «Implementaciones».

Multiplicación. Se deben implementar en C++ los algoritmos de multiplicación NAIVE y STRASSEN. Use la siguiente estructura de archivos.

- code/matrix_multiplication/algorithms/naive.cpp
- code/matrix_multiplication/algorithms/strassen.cpp

(2) **Mediciones de tiempo y memoria.** Implementar el programa que realizará las mediciones de tiempo y memoria en C++ (programas principales) y su respectivo makefile, que ejecutará los algoritmos y generará los archivos de salida en cada uno de los directorios measurements sorting y measurements matrix multiplication con los resultados de cada uno de los algoritmos.

- code/sorting/sorting.cpp
- code/matrix_multiplication/matrix_multiplication.cpp

(3) **Generación de gráficos.** Implementar el programa que generará los gráficos en PYTHON y que se encargará de leer los archivos generados por los programas principales guardados en measurements sorting y measurements matrix multiplication, para luego graficar los resultados obtenidos y guardarlos en formato PNG en plots sorting y plots matrix multiplication.

- code/sorting/scripts/plot_generator.py
- code/matrix_multiplication/scripts/plot_generator.py

(4) **Documentación.** Documentar cada uno de los pasos anteriores

- Completar el archivo README.md del directorio code
- Documentar en cada uno de sus programas, al inicio de cada archivo, fuentes de información, referencias y/o bibliografía utilizada para la implementación de cada uno de los algoritmos.

2.2. Mini-informe

Generar un mini-informe en $\LaTeX$ que contenga una introducción breve, los resultados obtenidos (tablas y gráficos) con una discusión sobre ellos, y una conclusión breve. En el material entregado podrá encontrar el Template en el directorio report que deben utilizar:

Template disponible en: report/ dentro del material entregado

Las indicaciones se encuentran en el archivo README.md y en la plantilla de $\LaTeX$.

3. Condiciones de entrega

- (1) La tarea es individual, sin excepciones.
- (2) La entrega debe realizarse vía aula.usm.cl, subiendo un archivo .zip que contenga todo el contenido de su proyecto (código, informe, datos, gráficos, etc.).
- (3) Si se utiliza algún código, idea, o contenido extraído de otra fuente, este debe ser citado en el lugar exacto donde se utilice.
- (4) Asegúrese que todas sus entregas tengan sus datos completos: número de la tarea, ramo, semestre, nombre y rol. Puede incluirlas como comentarios en sus fuentes $\LaTeX$ (en $\TeX$ comentarios son desde % hasta el final de la línea) o en posibles programas. Anótese como autor de los textos.
- (5) Si usa material adicional al discutido en clases, detállelo. Agregue información suficiente para ubicar ese material (en caso de no tratarse de discusiones con compañeros de curso u otras personas).
- (6) La fecha límite de entrega es el día 9 de Abril de 2026.
- (7) Nos reservamos el derecho de llamar a interrogación sobre algunas de las tareas entregadas. En tal caso, la nota de la tarea será la obtenida en la interrogación.

NO SE ACEPTARÁN TAREAS FUERA DE PLAZO.

NO PRESENTARSE A UN LLAMADO A INTERROGACIÓN SIN JUSTIFICACIÓN PREVIA SIGNIFICA AUTOMÁTICAMENTE NOTA 0.

A. Casos de prueba

A.1. Ordenamiento de un arreglo unidimensional de números enteros

Entrada: Leer un arreglo unidimensional A desde el archivo $\{n\}_{\{t\}}_{\{d\}}_{\{m\}}.txt$.

- n hace referencia a la cantidad de elementos (o largo del arreglo) y pertenece al conjunto $N = \{10^1, 10^3, 10^5, 10^7\}$.
- t hace referencia al tipo de matriz, y pertenece al conjunto $T = \{\text{ascendente}, \text{descendente}, \text{aleatorio}\}$.
- d hace referencia al conjunto dominio de cada elemento del arreglo. $d = \{D1, D7\}$, donde $D7$ implica que el dominio es $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ y $D1$ que el dominio es $\{0, 1, 2, 3, \dots, 10^7\}$.
- m hace referencia a la muestra aleatoria (o caso de prueba) y pertenece al conjunto $M = \{a, b, c\}$.

Salida: Escribir la matriz resultante $M_1 \times M_2$ en el archivo $\{n\}_{\{t\}}_{\{d\}}_{\{m\}}_{out}.txt$.

Los programas que generan estos arreglos se encuentran en

`code/sorting/scripts/array_generator.py`

A.2. Multiplicación de matrices cuadradas de números enteros

Entrada: Leer dos matrices cuadradas de entrada M_1 y M_2 desde los archivos $\{n\}_{\{t\}}_{\{d\}}_{\{m\}}_1.txt$ y $\{n\}_{\{t\}}_{\{d\}}_{\{m\}}_2.txt$, respectivamente.

- n hace referencia a la dimensión de la matriz (n filas y n columnas) y pertenece al conjunto $N = \{2^4, 2^6, 2^8, 2^{10}\}$.
- t hace referencia al tipo de matriz, y pertenece al conjunto $T = \{\text{dispersa}, \text{diagonal}, \text{densa}\}$.
- d hace referencia al dominio de cada coeficiente de la matriz $d = \{D0, D10\}$, donde $D0$ implica que el dominio es $\{0, 1\}$ y $D10$ que el dominio es $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.
- m hace referencia a la muestra aleatoria (o caso de prueba) y pertenece al conjunto $M = \{a, b, c\}$.

Salida: Escribir la matriz resultante $M_1 \times M_2$ en el archivo $\{n\}_{\{t\}}_{\{d\}}_{\{m\}}_{out}.txt$.

Los programas que generan estas matrices se encuentran en

`code/matrix_multiplication/scripts/matrix_generator.py`